

Python Programming Fundamentals for Deep Learning

Instructor: Samaher Alzahrani

Date: May 13, 2026

Class 3: Build Neural Networks in Python

Topics Covered in This Class:

- Introduction to Deep Learning.
- Training Loops.
- Gradients.
- Overfitting and Underfitting.
- Introduction to PyTorch Library.
- Tensors.
- Building Fully-Connected Neural Network.
- Classification Neural Network.
- Example: Linear Regression Neural Network.



Introduction to PyTorch

PyTorch is an open-source machine learning library. It provides powerful tools for numerical computation using tensors and for building and training deep learning models.



PyTorch is widely used in both research and industry because of its simple Python-based interface, dynamic computation graphs, and strong support for GPU acceleration.

PyTorch is organized into several core components:

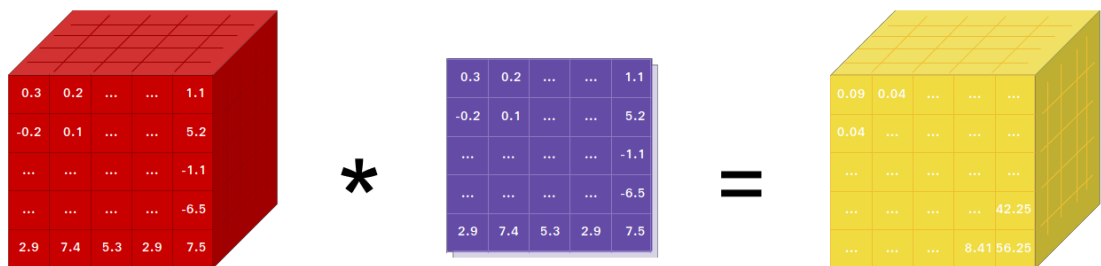
- `torch` : Tensor operations and automatic differentiation.
- `torch.nn` : Neural network layers and loss functions.
- `torch.optim` : Optimization algorithms.
- `torch.utils.data` : Data loading and preprocessing utilities.

Before building deep learning models, it is important to understand tensors.

Tensors

A PyTorch `tensor` is the fundamental data structure in the PyTorch library, designed as a multi-dimensional array similar to a NumPy `ndarray`. tensors are used to encode model inputs, outputs, and parameters.

The primary difference between NumPy arrays and tensors is that tensors are designed specifically for deep learning, offering GPU acceleration and automatic differentiation, whereas NumPy arrays are built for general-purpose scientific computing on CPUs.



To learn more about tensors and tensor decomposition see:

- [PyTorch Documentation](#)
- [Tensor Decomposition and Application](#)

```
In [28]: import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch
import pandas as pd
import torch.optim as optim
```

Load Tensor from List

```
In [29]: List = [
    [1, 3, 6],
    [2, 4, 2]
]
tensor_1 = torch.tensor(List)
tensor_1
```

```
Out[29]: tensor([[1, 3, 6],
                [2, 4, 2]])
```

Load Tensor from Numpy Array

```
In [30]: array = np.array(List)
         tensor_2 = torch.tensor(array)
         tensor_2
```

```
Out[30]: tensor([[1, 3, 6],
                 [2, 4, 2]])
```

```
In [31]: print(tensor_1.shape)
         print(tensor_1.dtype)
         print(tensor_1.device)
```

```
torch.Size([2, 3])
torch.int64
cpu
```

Deep learning often requires a GPU, which can offer:

- Parallel computing capabilities.
- Faster training time.
- Better performance.

If we need to run our training in GPU, we need to set device as `cuda`.

```
In [32]: tensor_3 = torch.tensor(array, dtype= torch.float32, device= 'cuda', requires_grad= True)
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[32], line 1
----> 1 tensor_3 = torch.tensor(array, dtype= torch.float32, device= 'cuda', requires_grad= True)

File ~\AppData\Roaming\Python\Python312\site-packages\torch\cuda\__init__.py:310,
in _lazy_init()
    305     raise RuntimeError(
    306         "Cannot re-initialize CUDA in forked subprocess. To use CUDA with
    "
    307         "multiprocessing, you must use the 'spawn' start method"
    308     )
    309 if not hasattr(torch._C, "_cuda_getDeviceCount"):
--> 310     raise AssertionError("Torch not compiled with CUDA enabled")
    311 if _cudart is None:
    312     raise AssertionError(
    313         "libcudart functions unavailable. It looks like you have a broken
    build?"
    314     )

AssertionError: Torch not compiled with CUDA enabled
```

```
In [ ]: tensor_3 = torch.tensor(array, dtype= torch.float32, requires_grad= True)
         print(tensor_3.requires_grad)
         print(tensor_3.dtype)
```

```
True
torch.float32
```

In standard PyTorch training, model parameters created by `torch.nn` modules (such as `nn.Linear`) automatically have `requires_grad=True`, so there is usually no need to set this argument manually.

We need to set the tensor argument `requires_grad=True` explicitly only when we want to compute gradients with respect to tensors other than the model parameters. This is useful in mathematical and scientific applications where derivatives of the model output with respect to the input variables are required.

To learn more about automatic differentiation see:

- [Reversed Automatic Differentiation \(VJP\)](#)
- [Forward Automatic Differentiation \(JVP\)](#)

Common Initialization Methods

```
In [37]: tensor_4 = torch.empty(size= (3,3))
tensor_4
```

```
Out[37]: tensor([[0.5000, 1.0000, 1.5000],
                [2.0000, 2.5000, 3.0000],
                [3.5000, 4.0000, 4.5000]])
```

```
In [38]: tensor_5 = torch.empty(size= (3,3))
tensor_5
```

```
Out[38]: tensor([[0.2191, 0.1745, 0.3092],
                [0.9526, 0.9833, 0.0153],
                [0.5251, 0.6287, 0.4756]])
```

```
In [34]: tensor_6 = torch.ones((3,3))
tensor_6
```

```
Out[34]: tensor([[1., 1., 1.],
                [1., 1., 1.],
                [1., 1., 1.]])
```

```
In [39]: tensor_7 = torch.eye(3,3)
tensor_7
```

```
Out[39]: tensor([[1., 0., 0.],
                [0., 1., 0.],
                [0., 0., 1.]])
```

```
In [40]: tensor_8 = torch.rand((3,3))
tensor_8
```

```
Out[40]: tensor([[0.3124, 0.5843, 0.4502],
                [0.3626, 0.2554, 0.0326],
                [0.0577, 0.3179, 0.8948]])
```

```
In [41]: tensor_9 = torch.arange(0, 5, 0.5)
```

```
tensor_9
```

```
Out[41]: tensor([0.0000, 0.5000, 1.0000, 1.5000, 2.0000, 2.5000, 3.0000, 3.5000, 4.0000,
               4.5000])
```

```
In [42]: tensor_10 = torch.linspace(0, 5, 5)
tensor_10
```

```
Out[42]: tensor([0.0000, 1.2500, 2.5000, 3.7500, 5.0000])
```

Tensor Operation

```
In [44]: tensor_11 = tensor_6 + tensor_8
tensor_11
```

```
Out[44]: tensor([[1.3124, 1.5843, 1.4502],
                 [1.3626, 1.2554, 1.0326],
                 [1.0577, 1.3179, 1.8948]])
```

```
In [45]: tensor_12 = tensor_6 * tensor_7
tensor_12
```

```
Out[45]: tensor([[1., 0., 0.],
                 [0., 1., 0.],
                 [0., 0., 1.]])
```

Building a Fully Connected Neural Network

Before building the network, we need to examine the shape of the input tensors. In PyTorch, a fully connected layer (`nn.Linear`) expects the input to have shape `(N, d)`, where:

- `N` is the number of samples (batch size),
- `d` is the number of input features.

For a single sample with `N` features, the input tensor must therefore have shape `(1, N)`.

Here, `array_1` is of shape `(4, 1)`. In this situation, we say, the input is of one feature and 4 samples.

```
In [50]: array_1 = np.array([
           [1.3],
           [3],
           [2],
           [1]
        ])
array_1.shape
```

```
Out[50]: (4, 1)
```

```
In [53]: array_2 = np.array([1, 2, 3, 4])  
array_2.shape
```

```
Out[53]: (4,)
```

```
In [51]: tensor_13 = torch.tensor(array_1)  
tensor_13.shape
```

```
Out[51]: torch.Size([4, 1])
```

Tensor_14 is of shape (4, 1). That is, it refers to four input samples with one feature. Whereas Tensor_15 is of shape (4,), which represents one input sample of a four features.

```
In [54]: tensor_14 = torch.tensor(array_1)  
tensor_14.shape
```

```
Out[54]: torch.Size([4, 1])
```

```
In [55]: tensor_15 = torch.tensor(array_2)  
tensor_15.shape
```

```
Out[55]: torch.Size([4])
```

Squeeze Method

The `squeeze` method is used to remove dimensions of size 1 from a tensor. It can be used to convert a tensor of shape (N, 1) into a one-dimensional tensor of shape (N,).

```
In [57]: tensor_14.squeeze().shape
```

```
Out[57]: torch.Size([4])
```

Reshape Method

The `reshape` method is used to change the shape of a tensor without changing its data. It is commonly used to convert a one-dimensional tensor of shape (N,) into a two-dimensional tensor of shape (N, 1).

```
In [58]: tensor_15.reshape(-1,1)
```

```
Out[58]: tensor([[1],  
                [2],  
                [3],  
                [4]])
```

Now, we are ready to build our network!

```
In [ ]: input_tensor = torch.tensor(  
        [[1.3, 4.5, 9]]  
    )
```

```
Out[ ]: torch.Size([1, 3])
```

Linear layers are the building blocks for any network. A linear layer would take the input, perform a linear transformation for it, and returns output.

```
In [60]: Linear_Layer = nn.Linear(in_features=3, out_features=2)  
output = Linear_Layer(input_tensor)  
output
```

```
Out[60]: tensor([[ -2.5843,  0.6167]], grad_fn=<AddmmBackward0>)
```

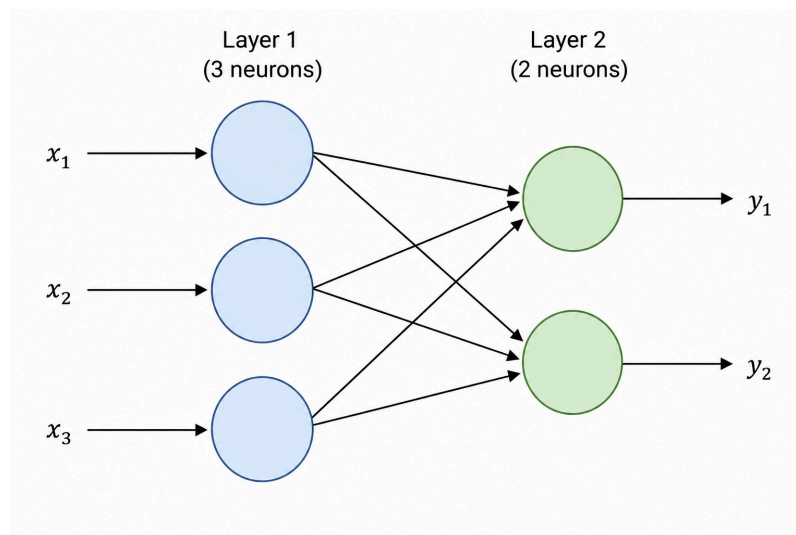
Each linear layer has weight and bias assigned automatically for it. We use the `.weight` and `.bias` methods to display them.

```
In [61]: Linear_Layer.weight
```

```
Out[61]: Parameter containing:  
tensor([[ -0.2114, -0.0600, -0.2858],  
        [ -0.1944, -0.5303,  0.3547]], requires_grad=True)
```

```
In [62]: Linear_Layer.bias
```

```
Out[62]: Parameter containing:  
tensor([0.5326, 0.0639], requires_grad=True)
```



Now, to build a model we often need to apply multiple linear layers to the input. To do this, we use `nn.Sequential()` to stack layers in order.

```
In [67]: model = nn.Sequential(  
    nn.Linear(3,15),  
    nn.Linear(15,20),  
    nn.Linear(20, 2)  
)
```

Input is passed through the linear layers. The input has 3 features and the output has 2 features.

```
In [68]: input_tensor
```

```
Out[68]: tensor([[1.3000, 4.5000, 9.0000]])
```

```
In [70]: output_tensor = model(input_tensor)  
output_tensor
```

```
Out[70]: tensor([[0.1667, 1.1050]], grad_fn=<AddmmBackward0>)
```

Classification Neural Network

```
In [ ]: df1 = pd.DataFrame(  
    {  
        'Feature_1': [ 0.5123, -1.2045, 1.3345, -1.1204, 0.2874],  
        'Feature_2': [-0.7421, 0.9184, -0.4452, 0.6637, 1.5520],  
        'Feature_3': [ 0.3341, -1.5532, 0.7741, -0.2185, -0.8841],  
        'Feature_4': [ 1.1207, 0.2876, -0.9923, 0.4419, -0.3367]  
    }, index= ['Sample_1', 'Sample_2', 'Sample_3', 'Sample_4', 'Sample_5']  
)  
  
df1
```

```
Out[ ]:
```

	Feature_1	Feature_2	Feature_3	Feature_4
Sample_1	0.5123	-0.7421	0.3341	1.1207
Sample_2	-1.2045	0.9184	-1.5532	0.2876
Sample_3	1.3345	-0.4452	0.7741	-0.9923
Sample_4	-1.1204	0.6637	-0.2185	0.4419
Sample_5	0.2874	1.5520	-0.8841	-0.3367

```
In [75]: input_data = torch.tensor(  
  
    [ 0.5123, -0.7421, 0.3341, 1.1207],  
    [-1.2045, 0.9184, -1.5532, 0.2876],  
    [ 1.3345, -0.4452, 0.7741, -0.9923],  
    [-1.1204, 0.6637, -0.2185, 0.4419],  
    [ 0.2874, 1.5520, -0.8841, -0.3367]  
  
)
```



```
In [77]: classification_model = nn.Sequential(
    nn.Linear(4,10),
    nn.Linear(10,1),
    nn.Sigmoid() # Activation function.
)

classification_output = classification_model(input_data)
classification_output
```

```
Out[77]: tensor([[0.5785],
    [0.5675],
    [0.5093],
    [0.5563],
    [0.4912]], grad_fn=<SigmoidBackward0>)
```

Sigmoid and Softmax Activation Functions

Sigmoid Function

The sigmoid function is an activation function that maps any real-valued number into the range $(0, 1)$. It is commonly used in binary classification problems because it can be interpreted as a probability.

Mathematically, it is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Softmax Function

The softmax function is a generalization of the sigmoid function used for multi-class classification problems. It converts a vector of raw scores (logits) into a probability distribution.

Mathematically, for a vector (x_i) :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Function	Output Range	Use Case
Sigmoid	$(0, 1)$	Binary classification
Softmax	$(0, 1)$, sum = 1	Multi-class classification

Linear Regression

For a solid introduction to the theory of least squares regression, see:

- [Ron Larson - Elementary Linear Algebra.pdf](#) - see page 99.

An implimentation of the example using algebra not the model

- [The Study of Gold Prices During the Year 2022.](#)

Now, this is the data we use.

```
In [79]: df2 = pd.DataFrame({
    "Month": [
        "January", "February", "March", "April",
        "May", "June", "July", "August",
        "September", "October", "November", "December"
    ],
    "Gold_Price_Avg_SAR": [
        6813.58, 6980.39, 7321.63, 7263.27,
        6937.15, 6899.42, 6533.16, 6628.29,
        6330.36, 6262.90, 6504.13, 6756.67
    ]
})

df2
```

```
Out[79]:
```

	Month	Gold_Price_Avg_SAR
0	January	6813.58
1	February	6980.39
2	March	7321.63
3	April	7263.27
4	May	6937.15
5	June	6899.42
6	July	6533.16
7	August	6628.29
8	September	6330.36
9	October	6262.90
10	November	6504.13
11	December	6756.67

```
In [82]: x = torch.tensor([[0.0], [1.0], [2.0], [3.0],
    [4.0], [5.0], [6.0], [7.0],
    [8.0], [9.0], [10.0], [11.0]])

y = torch.tensor(df2["Gold_Price_Avg_SAR"].values,
    dtype=torch.float32).view(-1, 1)

# Define the linear regression model (one linear layer model).
# y = w*x + b

model = nn.Linear(in_features=1, out_features=1)
```

```

# Loss function: Mean Squared Error
criterion = nn.MSELoss()

# Optimizer: Adam
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# Training Loop
epochs = 1000

for epoch in range(epochs):

    # Forward pass
    y_pred = model(x)

    # Compute Loss
    loss = criterion(y_pred, y)

    # Clear old gradients
    optimizer.zero_grad()

    # Compute gradients
    loss.backward()

    # Update parameters
    optimizer.step()

    # Print progress every 200 epochs
    if (epoch + 1) % 200 == 0:
        print(f"Epoch {epoch+1}, Loss = {loss:.2f}")

# -----
# Display Learned equation
# -----
w = model.weight.item()
b = model.bias.item()

print(f"\nLearned model:")
print(f"Price ≈ {w:.2f} × Month + {b:.2f}")

```

Epoch 200, Loss = 1565451.12

Epoch 400, Loss = 216960.83

Epoch 600, Loss = 71814.45

Epoch 800, Loss = 56191.23

Epoch 1000, Loss = 54509.68

Learned model:

Price ≈ -60.42 × Month + 7094.16

Plot Result

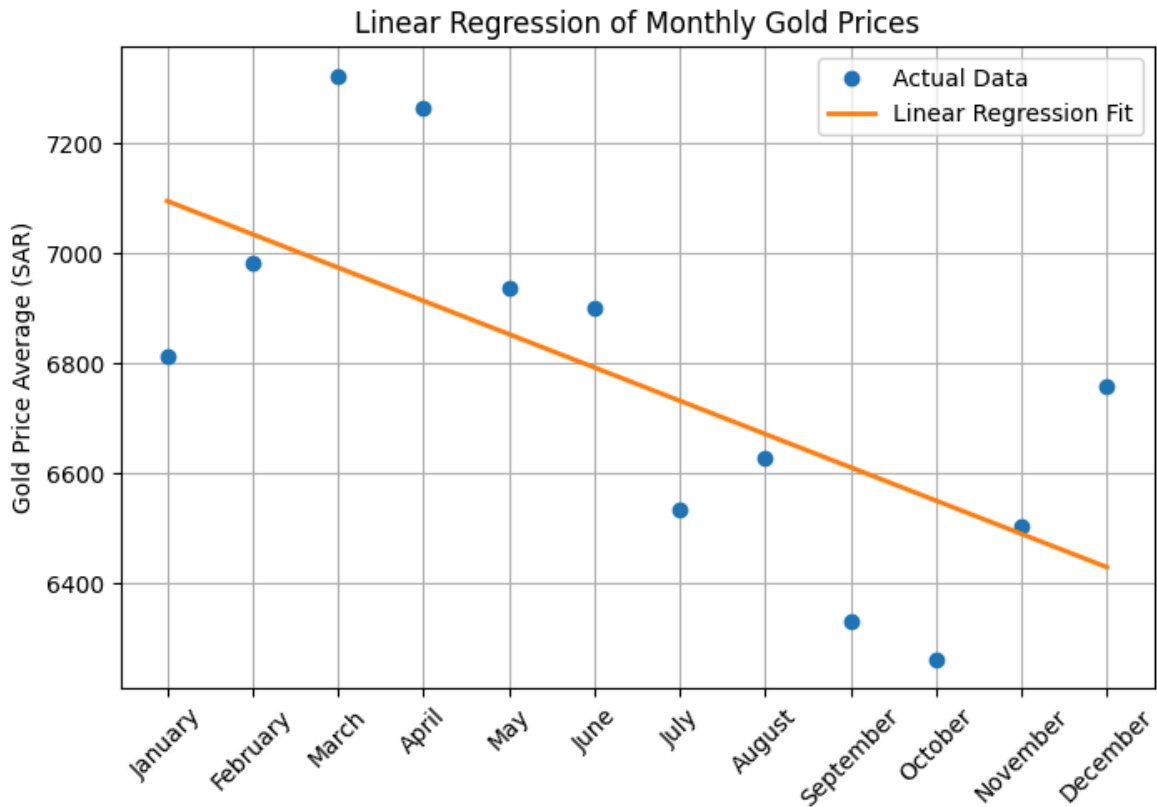
```

In [86]: # Obtain Prediction
with torch.no_grad():
    y_fit = model(x)

# Plot results
plt.figure(figsize=(8, 5))
plt.plot(df2["Month"], y.numpy(), 'o', label="Actual Data")

```

```
plt.plot(df2["Month"], y_fit.numpy(), '-', linewidth=2,
         label="Linear Regression Fit")
plt.xticks(rotation=45)
plt.ylabel("Gold Price Average (SAR)")
plt.title("Linear Regression of Monthly Gold Prices")
plt.legend()
plt.grid(True)
plt.show()
```



The Sum of Squared Error

```
In [87]: E = model(x) - y
         print(E)
```

```
tensor([[ 280.5791],
        [  53.3481],
        [-348.3130],
        [-350.3740],
        [ -84.6748],
        [-107.3657],
        [ 198.4727],
        [  42.9219],
        [ 280.4312],
        [ 287.4702],
        [ -14.1812],
        [-327.1421]], grad_fn=<SubBackward0>)
```

```
In [90]: E.T @ E
```

```
Out[90]: tensor([[654089.2500]], grad_fn=<MmBackward0>)
```

Kindly Answer the Survey

